

Дизайн и анализ на алгоритми упражнение 4

Станислав Игов (Стъката)

2025/2026

Сортиране. Алгоритми за сортиране. Задачи 2Sum, 3Sum, MinAbsSubarray, Merge Intervals

Сортиране

Задача за сортиране

Нека е дадена наредена n -торка

$$A = (a_1, a_2, \dots, a_n).$$

Задачата за сортиране се състои в това да се намери такава наредена n -торка

$$A' = (a'_1, a'_2, \dots, a'_n),$$

която е пермутация на елементите на A и удовлетворява условието за подреденост

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

(при сортиране във възходящ ред).

Предполага се, че елементите са сравними помежду си, т.е. върху множеството от стойности е дефинирана операция за сравнение (например $\leq$, $\geq$), чрез която може да се определи относителният ред на всеки два елемента.

С други думи, входът на задачата е наредена n -торка от сравними елементи, а изходът е същите елементи, но подредени според избраното отношение на наредба.

Защо сортираме?

Сортирането подрежда данните според дадено отношение на наредба (например във възходящ или низходящ ред), което улеснява тяхната обработка.

Подредените данни позволяват по-ефективно търсене, по-лесно намиране на минимални и максимални стойности, както и по-удобен анализ на информацията. Например върху сортирани данни може да се използва бърз алгоритъм като бинарно търсене.

Освен това сортирането е важна стъпка в много други алгоритми и приложения, поради което е една от най-фундаменталните задачи в алгоритмите и информатиката.

Алгоритми за сортиране

- $O(n^2)$ в най-лошия случай
 - SELECTIONSORT
 - INSERTIONSORT
 - QUICKSORT (при случаен *pivot*, средна сложност $O(n \log n)$)
- $O(n \log n)$
 - HEAPSORT
 - MERGESORT
 - QUICKSORT (при *pivot*, определен като “Median of medians”)
- $O(n + d) \rightarrow$ COUNTINGSORT (използваме, когато имаме горна граница d за елементите на масива)

Задача 2SUM

2PointersSum

Вход: Масив $A[1..n]$ от рационални числа, сортиран във възходящ ред и *target* - рационално число

Изход: true ако $\exists i, j : A[i] + A[j] = target$, false в противен случай

TwoPointersSum($A[1..n]$: array of int, *target*: int)

```
@1  l <- 1
@2  r <- n
@3  while l < r and A[l] + A[r] != target do
@4      if A[l] + A[r] < target then
@5          l++
@6      else
@7          r--
@8      endif
@9  endwhile
@10 return l < r
```

Нека l_i, r_i са стойностите на променливите l и r преди i -тата итерация на цикъла на ред @3.

Дефинираме инварианта:

$$I(i) \iff \text{Ако съществуват индекси } p < q \text{ с } A[p] + A[q] = target, \text{ то } l_i \leq p < q \leq r_i.$$

Еквивалентен запис с квантори:

$$I(i) \iff \exists p, q : A[p] + A[q] = target \implies l_i \leq p < q \leq r_i$$

Тоест: ако решение съществува, то се намира в текущия интервал $[l_i, r_i]$.

Доказателство

База

Преди първата итерация имаме

$$l_1 = 1, \quad r_1 = n.$$

Ако съществуват индекси $p < q$ с $A[p] + A[q] = target$, очевидно

$$1 \leq p < q \leq n.$$

Следователно

$$l_1 \leq p < q \leq r_1,$$

което означава, че $I(1)$ е вярно.

Поддръжка

Да допуснем, че $I(i)$ е вярно в началото на i -тата итерация.

Разглеждаме стойността $A[l_i] + A[r_i]$.

Случай 1: $A[l_i] + A[r_i] < target$. Понеже масивът е сортиран възходящо, за всеки $j \leq r_i$ е изпълнено

$$A[l_i] + A[j] \leq A[l_i] + A[r_i] < target.$$

Следователно индексът l_i не може да участва в решение. Затова алгоритъмът увеличава l :

$$l_{i+1} = l_i + 1, \quad r_{i+1} = r_i.$$

Ако съществува решение (p, q) , то не може да има $p = l_i$, следователно $p \geq l_i + 1$. Тогава

$$l_{i+1} \leq p < q \leq r_{i+1}.$$

Следователно $I(i + 1)$ е вярно.

Случай 2: $A[l_i] + A[r_i] > target$. От сортираността следва, че за всеки $i \geq l_i$ е изпълнено

$$A[i] + A[r_i] \geq A[l_i] + A[r_i] > target.$$

Следователно индексът r_i не може да участва в решение. Алгоритъмът намалява r :

$$l_{i+1} = l_i, \quad r_{i+1} = r_i - 1.$$

Ако съществува решение (p, q) , то не може да има $q = r_i$, следователно $q \leq r_i - 1$ и

$$l_{i+1} \leq p < q \leq r_{i+1}.$$

Следователно $I(i + 1)$ е вярно.

Така инвариантът се поддържа.

Терминация

Цикълът приключва когато условието на ред 03 стане лъжа. Това става в два случая:

- $A[l] + A[r] = target$ — тогава алгоритъмът връща **true**;
- $l \geq r$ — интервалът вече не съдържа два различни индекса.

От инварианта следва, че ако решение съществува, то трябва да се намира в интервала $[l, r]$. Когато $l \geq r$, такъв интервал не може да съдържа двойка $p < q$, следователно решение не съществува.

Тогава алгоритъмът връща **false**, което е коректно.

Следователно алгоритъмът **TwoSum** е коректен.

Времева сложност

На всяка итерация на цикъла правим едно от следните 2:

- увеличаваме l с 1
- намаляваме r с 1

В най-лошия случай когато не можем да намерим $target$ ще минем през всички елементи на масива, тоест алгоритъмът ще се изпълни n пъти. От тук $\text{Time}(\text{TwoSum}) \in O(n)$

2SUM

Вход: Масив $A[1..n]$ от рационални числа и $target$ - рационално число

Изход: true ако $\exists i, j : A[i] + A[j] = target$, false в противен случай

```
TwoSum(A[1..n]: array of int, target: int)
@1 MergeSort(A[1..n])
@2 return TwoPointersSum(A[1..n], target)
```

Коректността на TwoSum е директно следствие от **Коректността на TwoPointersSum**, тъй като подаденият на @2 масив е сортираният вход и е в сила, че такива индекси $1 \leq i, j \leq n$ има в A т.с.т.к. има такива индекси $1 \leq i', j' \leq n$ в сортираната пермутация A' на A .

Времева сложност

```
@1  $O(n \log n)$ 
@2  $O(n)$ 
 $O(n \log n) + O(n) = O(n \log n) \Rightarrow \text{Time}(\text{TwoSum}) \in O(n \log n)$ 
```

3SUM

Вход: Масив $A[1..n]$ от рационални числа- **Изход:** true ако $\exists i, j, k : A[i] + A[j] + A[k] = 0$, false в противен случай

```
ThreeSum(A[1..n]: array of int)
@1 MergeSort(A[1..n])
@2 i <- 1
@2 while i < n - 1 and not TwoPointersSum(A[i+1, n], -A[i])
@3     i++
@4 endwhile
@5 return i < n-1
```

Алтернативен начин на обосновка:

Алгоритъмът първо сортира масива. Това е важно, защото алгоритъмът TwoPointersSum работи коректно само върху сортиран масив.

След това алгоритъмът обхожда последователно всеки елемент $A[i]$, който разглежда като първи елемент на възможна тройка. За всеки такъв елемент задачата се свежда до следното: да се намерят два елемента в подмасива $A[i+1..n]$, чиято сума е $-A[i]$.

Точно това проверява извикването

$$\text{TwoPointersSum}(A[i+1..n], -A[i]).$$

Ако съществуват индекси $j, k > i$, за които

$$A[j] + A[k] = -A[i],$$

то следва

$$A[i] + A[j] + A[k] = 0,$$

следователно съществува тройка със сума 0 и алгоритъмът може да върне **true**.

Коректността следва директно от коректността на TwoPointersSum. Този алгоритъм е коректен и връща **true** точно тогава, когато в даден сортиран масив съществува двойка елементи с дадена сума.

За фиксиран индекс i алгоритъмът проверява всички двойки в подмасива $A[i+1..n]$ чрез TwoPointersSum.

Алгоритъмът последователно проверява всеки възможен избор на първи елемент на тройката ($i = 1, \dots, n-2$).

Следователно се разглеждат всички възможни тройки $i < j < k$.

Ако съществува тройка със сума 0, тя ще бъде намерена при итерацията със съответния i , защото TwoPointersSum

ще намери подходящите j, k .

Ако алгоритъмът не намери такава двойка за никой i , то такава тройка не съществува.

Следователно алгоритъмът връща **true** точно когато съществува тройка

$$A[i] + A[j] + A[k] = 0,$$

което доказва неговата коректност.

MinimumAbsSubarray

Да се намери инфиксна сума в масив с най-малка абсолютна стойност.

Вход: $A[1 \dots n]$ – масив от рационални числа.

Изход:

$$\min \left\{ \left| \sum_{k=i}^j A[k] \right| \mid 1 \leq i \leq j \leq n \right\}$$

– най-близка до 0 сума на непразен инфикс на A .

Лема Ако за $k < l$ имаме, че

$$|S_k - S_l| = \min\{|S_j - S_i| \mid 0 \leq i < j \leq n\},$$

то няма q , такова че $S_k < S_q < S_l$ или $S_l < S_q < S_k$.

Сега след Лемата остава да съобразим, че е достатъчно да сортираме префиксните суми и да търсим минимална абсолютна разлика между някои два последователни.

```
MinAbsSubarray(A[1..n] : array of rationals)
@1 P[0..n] <- malloc(n + 1) : ARRAY OF rationals
@2 P[0] <- 0
@3 for i = 1 to n do
@4     P[i] <- P[i - 1] + A[i]
@5 end for
@6 MergeSort(P[0..n])
@7 m <- +inf
@8 for i = 1 to n do
@9     m <- min{m, P[i] - P[i - 1]}
@10 end for
@11 return m
```

Нека m_i е състоянието на m преди i -тата итерация на цикъла на ред @8.

Дефинираме:

$$I(i) \iff m_i = \min_{1 \leq j < i} (P[j] - P[j - 1])$$

Доказателството на алгоритма става стандартно по индукция и ще го оставим за упражнение на читателя.

Сливане на интервали

Вход: Масив $I[1..n]$ от интервали, където всеки интервал е от вида $I[i] = [s_i, e_i]$ и $s_i \leq e_i$.

Изход: Масив R от интервали, получен чрез обединяване на всички припокриващи се интервали в I , така че:

- всеки интервал в R представя максимален обединен интервал,
- интервалите в R не се припокриват,
- $\bigcup_{i=1}^n I[i] = \bigcup_{j=1}^{|R|} R[j]$.

Функция на сравнението на интервали

Тъй като ще сортираме интервали е нужно да дефинираме начин по който ще ги сравняваме.
Нека I_1 и I_2 са интервали.

$$I_1 < I_2 \iff I_1.start < I_2.start \text{ or } I_1.start = I_2.start \text{ and } I_1.end < I_2.end$$

```
MergeIntervals(I[1..n]: array of intervals)
@1 MergeSort(I[1..n]) by start time
@2 R <- malloc(n)
@3 k <- 0
@4 for i <- 1 to n do
@5     if R is empty or I[i].start > R[k].end // R is empty <=> k = 0
@6         k++
@7         R[k] <- I[i]
@8     else
@9         R[k].end <- max(R[k].end, I[i].end)
@10    endif
@11 endfor
@12 return R[1..k]
```

Нека k_i и R_i са състоянията съответно на k и масива R преди i -тата итерация на цикъла на ред @5.
Дефинираме:

$$I(i) \Leftrightarrow \begin{cases} k_i = |Merge(I[1..i-1])| \\ R_i[1..k_i] = Merge(I[1..i-1]) \\ \forall j \in [1..k_i-1] : R_i[j].end < R_i[j+1].start \end{cases}$$

където $Merge(I[1..i-1])$ е множеството от интервали, получено след обединяване на всички припокриващи се интервали от $I[1..i-1]$.

Интуитивно:

- $k_i = |Merge(I[1..i-1])|$
 k е броят на интервалите в резултата след обработване на първите $i-1$ интервала.
- $R_i[1..k_i] = Merge(I[1..i-1])$
Масивът R съдържа точно обединените интервали на вече обработените.
- $\forall j \in [1..k_i-1] : R_i[j].end < R_i[j+1].start$
Интервалите в R са сортирани и неприпокриващи се.

Доказателство

База: $i = 1$

Преди първата итерация имаме $k = 0$ и $R = \emptyset$. Следователно $R_1[1..k_1] = \emptyset$.
От друга страна

$$Merge(I[1..0]) = \emptyset$$

Тогава:

$$k_1 = 0 = |Merge(I[1..0])|$$

$$R_1[1..k_1] = \emptyset = Merge(I[1..0])$$

Третото условие също е изпълнено тривиално, тъй като няма индекси $j \in [1..k_1-1]$.
Следователно $I(1)$ е вярно.

Поддръжка:

Допускаме, че $I(i)$ е вярно. Ще докажем, че тогава е вярно и $I(i+1)$.
Разглеждаме какво се случва при обработване на интервала $I[i]$.

Случай 1: условието на ред @6 е истина.

$$R \text{ е празен} \quad \text{или} \quad I[i].start > R[k].end$$

Случай 1.1: R е празен.

Тогава $k_i = 0$ и на редове @7–@8 се изпълнява

$$k \leftarrow 1, \quad R[1] \leftarrow I[i]$$

Следователно

$$R_{i+1}[1..1] = \{I[i]\}$$

и

$$Merge(I[1..i]) = \{I[i]\}$$

защото $I[1..i-1]$ е празно.

Следователно

$$k_{i+1} = 1 = |Merge(I[1..i])|$$

$$R_{i+1}[1..k_{i+1}] = Merge(I[1..i])$$

Третото условие е изпълнено тривиално, защото има само един интервал.

Случай 1.2: R не е празен и

$$I[i].start > R[k].end$$

Тогава новият интервал не се припокрива с последния интервал в R . На редове @7–@8 добавяме нов интервал:

$$k_{i+1} = k_i + 1$$

$$R_{i+1}[k_{i+1}] = I[i]$$

Понеже интервалите I са сортирани по начало, всички предишни интервали са с начало $\leq I[i].start$. От условието $I[i].start > R[k_i].end$ следва, че $I[i]$ не се припокрива с никой интервал от R_i .

Следователно:

$$Merge(I[1..i]) = Merge(I[1..i-1]) \cup \{I[i]\}$$

Така

$$k_{i+1} = |Merge(I[1..i])|$$

$$R_{i+1}[1..k_{i+1}] = Merge(I[1..i])$$

Освен това

$$R_{i+1}[k_i].end < R_{i+1}[k_i + 1].start$$

следователно интервалите остават неприпокриващи се.

Случай 2: условието на ред @6 е лъжа.

Тогава

$$I[i].start \leq R[k_i].end$$

т.е. интервалът $I[i]$ се припокрива с последния интервал в R .

На ред @10 се изпълнява

$$R[k_i].end \leftarrow \max(R[k_i].end, I[i].end)$$

Това точно съответства на обединението на двата интервала. Следователно новият последен интервал в R е

$$R_{i+1}[k_i] = Merge(R_i[k_i], I[i])$$

Така

$$Merge(I[1..i]) = Merge(I[1..i-1])$$

като само последният интервал е разширен.

Следователно:

$$k_{i+1} = k_i = |Merge(I[1..i])|$$

$$R_{i+1}[1..k_{i+1}] = Merge(I[1..i])$$

и тъй като се променя само края на последния интервал, редът и неприпокриването на интервалите се запазват.

Следователно $I(i+1)$ е вярно.

Терминация.

Цикълът завършва когато $i = n + 1$. По инварианта получаваме

$$R_{n+1}[1..k_{n+1}] = Merge(I[1..n])$$

и

$$k_{n+1} = |Merge(I[1..n])|$$

Следователно масивът $R[1..k]$ съдържа точно обединението на всички припокриващи се интервали от $I[1..n]$ и интервалите в него са сортирани и неприпокриващи се.

Тъй като алгоритъмът връща $R[1..k]$, резултатът съвпада с $Merge(I[1..n])$, което доказва коректността на алгоритъма.